САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №6

Выполнил:

Зотеев Максим

БР1.1

Проверил:
Добряков Д. И.

Санкт-Петербург

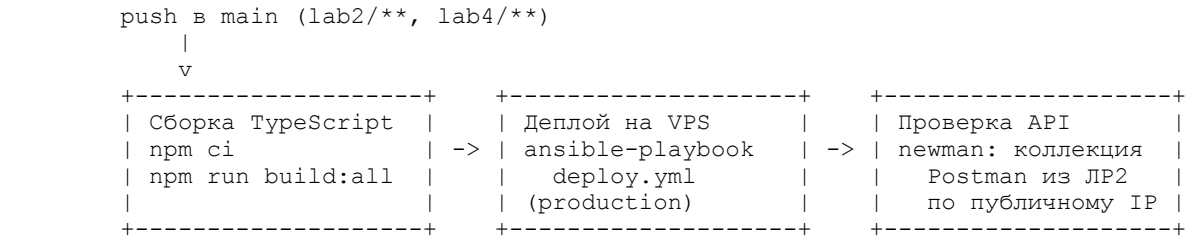
2026 г.

Задача
Необходимо настроить автодеплой (с триггером на обновление кода в репозитории, на определённой ветке) приложения на удалённый сервер с использованием GitHub Actions или GitLab CI.

Ход работы

1. Исходные данные и выбор подхода
Развёртывается микросервисная система сервиса аренды недвижимости из ЛР2–ЛР3 (api-gateway, identity-, property-, rental-, messaging-service, PostgreSQL и RabbitMQ) на тот же VPS, что и в ЛР4: Ubuntu 24.04, 82.115.222.101, 1 vCPU, 709 МиБ памяти. Код хранится в форке репозитория курса на GitHub (F0x1d/ITMO-ACS-Backend-2026-A), поэтому выбран GitHub Actions — он встроен в GitHub и не требует отдельного CI-сервера.
Логика развёртывания заново не пишется: в ЛР4 она уже описана Ansible-плейбуком deploy.yml, который идемпотентен (повторный запуск без изменений даёт changed=0) и сам определяет, какие сервисы пересобрать. Задача CI — по пушу проверить, что код собирается, запустить этот же плейбук с раннера GitHub вместо локальной машины и проверить развёрнутое API. Так ручной деплой и автодеплой остаются одним и тем же кодом.

2. Схема пайплайна
Workflow лежит в .github/workflows/rental-deploy.yml — GitHub ищет workflow только в этом каталоге в корне репозитория. Он запускается при пуше в ветку main, если изменились файлы lab2 (код приложения), lab4 (Ansible) или сам workflow; изменения отчётов (.docx, .pdf) деплой не запускают. Дополнительно workflow можно запустить вручную кнопкой Run workflow (workflow_dispatch). Пайплайн состоит из трёх последовательных заданий:



Если код не компилируется, деплой не начинается и сервер остаётся на прошлой версии. Задание деплоя привязано к окружению production, поэтому в интерфейсе GitHub видно, какая версия сейчас развёрнута и по какому адресу.
Дополнительные настройки workflow:
• if: github.repository == 'F0x1d/ITMO-ACS-Backend-2026-A' — файл попадёт и в основной репозиторий курса через Pull Request, где нет ни сервера, ни секретов; там задания пропускаются, а не падают;

• concurrency: rental-deploy без отмены — два быстрых пуша подряд не запускают плейбук одновременно, второй деплой ждёт окончания первого;
• permissions: contents: read — токену workflow не нужно ничего, кроме чтения кода;
• версии зафиксированы: Node.js 20 (как в Dockerfile сервисов), Python 3.13, ansible-core 2.21.5 и версии коллекций в requirements.yml — те же, что при локальном запуске в ЛР4.

3. Workflow

```
name: Деплой сервиса аренды (Зотеев Максим)

on:
  push:
    branches: [main]
    paths:
      - "БР1.1/Зотеев Максим/labs/lab2/**"
      - "БР1.1/Зотеев Максим/labs/lab4/**"
      - ".github/workflows/rental-deploy.yml"
      - "!**/*.docx"
      - "!**/*.pdf"
  workflow_dispatch:

# Деплои идут строго по очереди, новый не прерывает идущий.
concurrency:
  group: rental-deploy
  cancel-in-progress: false

permissions:
  contents: read

env:
  LAB2_DIR: БР1.1/Зотеев Максим/labs/lab2
  LAB4_DIR: БР1.1/Зотеев Максим/labs/lab4
  PUBLIC_URL: http://82.115.222.101

jobs:
  build:
    name: Сборка TypeScript
    # Файл попадёт и в основной репозиторий курса, где нет ни сервера, ни
секретов.
    if: github.repository == 'F0x1d/ITMO-ACS-Backend-2026-A'
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: 20
          cache: npm
          cache-dependency-path: ${{ env.LAB2_DIR }}/package-lock.json

      - name: Установка зависимостей
        working-directory: ${{ env.LAB2_DIR }}
        run: npm ci --no-audit --no-fund

      - name: Сборка всех сервисов
        working-directory: ${{ env.LAB2_DIR }}
        run: npm run build:all

  deploy:
    name: Деплой на VPS
    needs: build
    runs-on: ubuntu-24.04
    timeout-minutes: 60
```

```
    environment:
      name: production
      url: ${{ env.PUBLIC_URL }}/health
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-python@v7
        with:
          python-version: "3.13"

      - name: Установка Ansible и коллекций
        working-directory: ${{ env.LAB4_DIR }}
        run: |
          pip install ansible-core==2.21.5
          ansible-galaxy collection install -r requirements.yml

      - name: SSH-ключ, known_hosts и пароль vault
        working-directory: ${{ env.LAB4_DIR }}
        env:
          SSH_PRIVATE_KEY: ${{ secrets.SSH_PRIVATE_KEY }}
          SSH_KNOWN_HOSTS: ${{ secrets.SSH_KNOWN_HOSTS }}
          ANSIBLE_VAULT_PASSWORD: ${{ secrets.ANSIBLE_VAULT_PASSWORD }}
        run: |
          install -d -m 700 ~/.ssh
          install -m 600 /dev/null ~/.ssh/deploy_key
          printf '%s\n' "$SSH_PRIVATE_KEY" > ~/.ssh/deploy_key
          printf '%s\n' "$SSH_KNOWN_HOSTS" > ~/.ssh/known_hosts
          install -m 600 /dev/null .vault_pass
          printf '%s' "$ANSIBLE_VAULT_PASSWORD" > .vault_pass

      - name: Проверка синтаксиса плейбука
        working-directory: ${{ env.LAB4_DIR }}
        run: ansible-playbook deploy.yml --syntax-check

      - name: ansible-playbook deploy.yml
        working-directory: ${{ env.LAB4_DIR }}
        env:
          ANSIBLE_FORCE_COLOR: "1"
        run: ansible-playbook deploy.yml --private-key ~/.ssh/deploy_key

      - name: Удаление ключа и пароля vault
        if: always()
        working-directory: ${{ env.LAB4_DIR }}
        run: rm -f ~/.ssh/deploy_key .vault_pass

  smoke:
    name: Проверка API (newman)
    needs: deploy
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v7

      - name: Postman-коллекция против развёрнутого API
        working-directory: ${{ env.LAB2_DIR }}
        run: >-
          npx --yes newman@6 run postman/rental-api.lab2.postman_collection.json
          --env-var base_url=${{ env.PUBLIC_URL }}/api/v1
```

4. Доступ к серверу и секреты

Раннеру GitHub нужны три вещи: SSH-ключ для входа под deploy, ключи самого сервера для проверки его подлинности и пароль от Ansible Vault, которым зашифрованы секреты приложения. Они хранятся в зашифрованных секретах репозитория (Settings → Secrets and variables → Actions) и заданы командой gh secret set:

```
ANSIBLE_VAULT_PASSWORD  2026-10-17T13:11:35Z
SSH_KNOWN_HOSTS  2026-10-17T13:11:34Z
SSH_PRIVATE_KEY  2026-10-17T13:11:33Z
```

• SSH_PRIVATE_KEY — отдельный ключ ed25519 без пароля, созданный только для CI. Личный ключ разработчика в GitHub не передаётся, поэтому ключ CI можно отозвать, не трогая остальной доступ. После загрузки в GitHub локальная копия приватного ключа удалена. Публичная часть лежит в роли common и добавляется в authorized_keys пользователя deploy тем же deploy.yml, не заменяя личный ключ из bootstrap.yml:

```
- name: SSH-ключ GitHub Actions для deploy
  ansible.posix.authorized_key:
    user: "{{ deploy_user }}"
    key: "{{ lookup('ansible.builtin.file', 'github-actions.pub') }}"
```

• SSH_KNOWN_HOSTS — открытые ключи сервера (rsa, ecdsa, ed25519), взятые из локального known_hosts, где они были проверены при первом подключении в ЛР4. Проверка ключа хоста остаётся включённой (host_key_checking = True); получение ключей через ssh-keyscan прямо в CI приняло бы любой ключ, в том числе подменённый.

• ANSIBLE_VAULT_PASSWORD — пароль от group_vars/all/vault.yml. Сами секреты приложения (JWT_SECRET, пароли PostgreSQL и RabbitMQ) в GitHub не копируются: они остаются зашифрованными в репозитории, как и в ЛР4.

Секреты передаются в шаг через переменные окружения и записываются в файлы с правами 0600; последний шаг (if: always()) удаляет ключ и пароль даже при ошибке деплоя. В логах GitHub значения секретов заменяются на ***:

```
##[group]Run install -d -m 700 ~/.ssh
install -d -m 700 ~/.ssh
install -m 600 /dev/null ~/.ssh/deploy_key
printf '%s\n' "$SSH_PRIVATE_KEY" > ~/.ssh/deploy_key
printf '%s\n' "$SSH_KNOWN_HOSTS" > ~/.ssh/known_hosts
install -m 600 /dev/null .vault_pass
printf '%s' "$ANSIBLE_VAULT_PASSWORD" > .vault_pass
shell: /usr/bin/bash -e {0}
env:
  LAB2_DIR: БР1.1/Зотеев Максим/labs/lab2
  LAB4_DIR: БР1.1/Зотеев Максим/labs/lab4
  PUBLIC_URL: http://82.115.222.101
  SSH_PRIVATE_KEY: ***
  SSH_KNOWN_HOSTS: ***
  ANSIBLE_VAULT_PASSWORD: ***
```

5. Доработка плейбука ЛР4 для запуска из CI

В ЛР4 код копировался на сервер rsync'ом, а список изменённых файлов из его вывода определял, какие образы пересобрать. rsync по умолчанию считает файл изменённым, если отличаются размер или время изменения. На локальной машине время изменения файлов стабильно, но раннер GitHub каждый раз делает свежий git checkout, и у всех файлов оно новое — каждый деплой пересобирал бы все пять образов, что на одном vCPU занимает десятки минут. Поэтому сравнение переведено на содержимое файлов, а время изменения, владелец и группа больше не переносятся:

```
- name: Копирование кода lab2 на сервер
  ansible.posix.synchronize:
    src: "{{ app_src }}"
```

```
      dest: "{{ app_dir }}/"
      delete: true
      checksum: true
      times: false
      owner: false
      group: false
```

Дополнительно при разборе вывода rsync отбрасываются строки, начинающиеся с «.» — так rsync помечает файлы, у которых изменились только атрибуты (например, права), а содержимое прежнее. Пересборку вызывают только реально изменённые, новые и удалённые файлы.

Перед первым пушем это проверено локально в условиях CI: из git-архива распакована свежая копия lab2 с новым временем изменения всех файлов, ssh-agent отключён, вход только по ключу CI. Плейбук не пересобрал ни одного образа и ничего не изменил на сервере:

```
rental-vps : ok=33 changed=0 unreachable=0 failed=0 skipped=10
```

6. Проверка

Workflow запущен пушем коммита в main. Все три задания выполнились успешно, весь пайплайн занял около трёх минут:

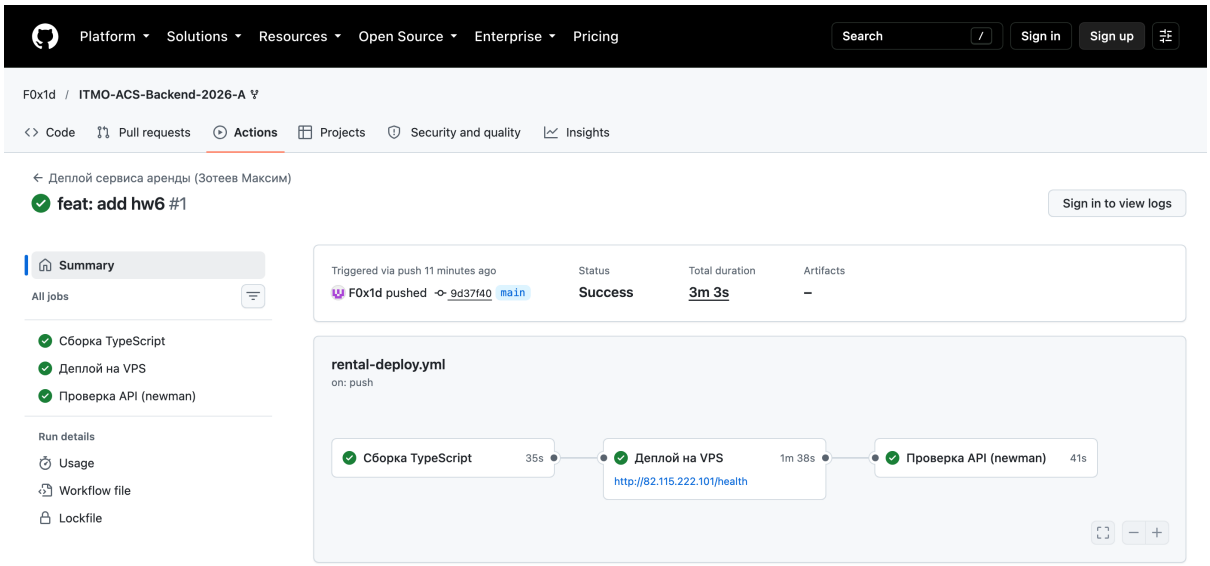


Рисунок 1 — Запуск workflow в GitHub Actions

Задание «Сборка TypeScript» устанавливает зависимости по package-lock.json и компилирует общий пакет и все пять сервисов:

```
added 809 packages in 13s

> build:all
> npm run build --workspaces --if-present
> @rental/shared@1.0.0 build
> tsc
> api-gateway@1.0.0 build
> tsc
> identity-service@1.0.0 build
> tsc
> messaging-service@1.0.0 build
> tsc
> property-service@1.0.0 build
> tsc
```

```
> rental-service@1.0.0 build
> tsc
```

Задание «Деплой на VPS» подключилось к серверу по ключу CI с проверкой ключа хоста и выполнило deploy.yml. Код в этом коммите не менялся, поэтому плейбук не пересобрал ни одного образа и не тронул контейнеры; последняя задача проверила http://82.115.222.101/health уже с раннера GitHub, то есть снаружи, через nginx:

```
TASK [app : Копирование кода lab2 на сервер]
ok: [rental-vps]

TASK [app : Сервисы, которые нужно пересобрать]
ok: [rental-vps]

TASK [app : Сборка образа сервиса]
skipping: [rental-vps]

TASK [app : Запуск контейнеров]
ok: [rental-vps]

TASK [app : api-gateway отвечает]
ok: [rental-vps]

TASK [API доступно снаружи через nginx]
ok: [rental-vps -> localhost]

PLAY RECAP
rental-vps : ok=33 changed=0 unreachable=0 failed=0 skipped=10
```

Задание «Проверка API» прогнало Postman-коллекцию из ЛР2 против развёрнутого приложения по публичному адресу — регистрация, авторизация, объекты, аренды, сообщения и проверки прав доступа:

	executed	failed
iterations	1	0
requests	62	0
test-scripts	62	0
prerequest-scripts	64	0
assertions	127	0
total run duration: 23.5s		
total data received: 25.2kB (approx)		
average response time: 338ms [min: 199ms, max: 2.1s, s.d.: 302ms]		

Заключение

В рамках ДЗ6 настроен автоматический деплой сервиса аренды недвижимости средствами GitHub Actions: пуш в ветку main с изменениями кода приложения или конфигурации развёртывания запускает пайплайн, который собирает TypeScript-код всех сервисов, разворачивает приложение на VPS Ansible-плейбуком из ЛР4 и проверяет развёрнутое API Postman-коллекцией. Для CI выделен отдельный SSH-ключ, подлинность сервера проверяется по заранее известным ключам, а секреты хранятся в секретах GitHub и Ansible Vault и не попадают в логи. Плейбук доработан под свежий checkout на раннере: изменения определяются по содержимому файлов, поэтому

пересобираются только затронутые сервисы, а запуск без изменений кода ничего не меняет на сервере. Прогон пайплайна завершился успешно: 62 запроса и 127 проверок без ошибок.